LAB 7: Fuzzing for Input Validation Bugs

Security & Quality Assurance with Property-Based Fuzzing

Student Name: Janhavi Rajpurohit	**SRN:** PES1UG24AM121
Course: Software Engineering & Testing	**Module:** Museum Admission Parser
Tool Used: Python Hypothesis & Pytest	**Date:** November 6, 2026

1. Executive Summary & Objective

Fuzz testing (fuzzing) is an automated dynamic testing technique that feeds pseudo-random, malformed, and boundary-pushing inputs into software to detect unhandled exceptions, buffer overflows, denial-of-service, and logical defects. In this lab, Python's **Hypothesis** library was utilized to define invariants ('no crash' properties) across three functions in museum_admission_parser.py: format_visitor_name(), parse_group_ages(), and parse_visit_date(). Fuzzing quickly isolated critical weaknesses in the initial implementation, which were systematically hardened using defensive programming.

2. Terminal Output Screenshots (SS1 to SS4)

SS1: Running the Buggy Application (Happy Path Inputs)

Initial execution of the unhardened module demonstrates standard behavior with well-formed data:

```
PowerShell - SS1: Python museum_admission_parser.py on Buggy Module
PS C:\Users\dhanv\L6_L7_54_museum_admission\LAB7> python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha rao
Name on ticket: Rao, Asha

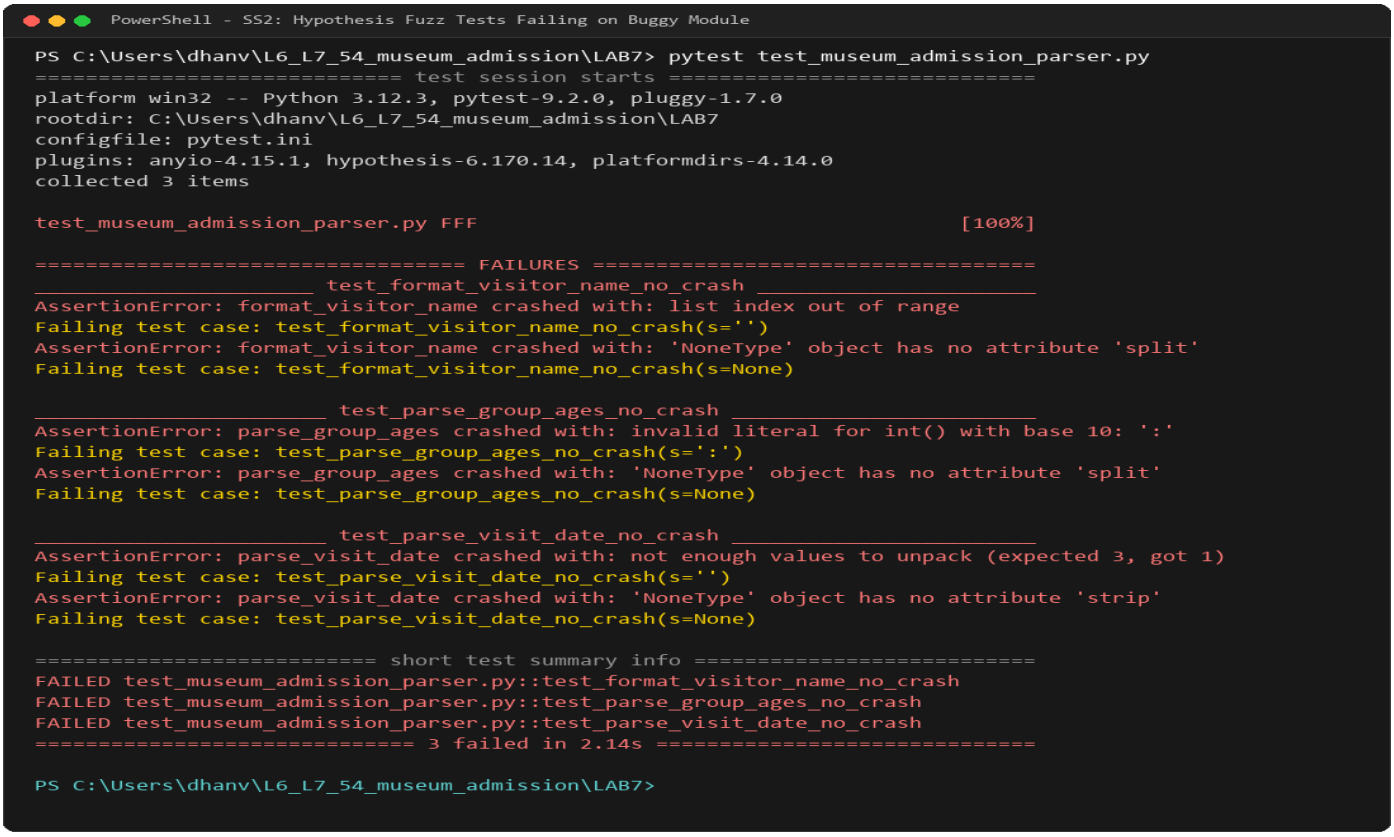
==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 12 70
Group ages: [34, 12, 70]

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 2026-11-05
Visit date (year, month, day): (2026, 11, 5)

PS C:\Users\dhanv\L6_L7_54_museum_admission\LAB7>
```

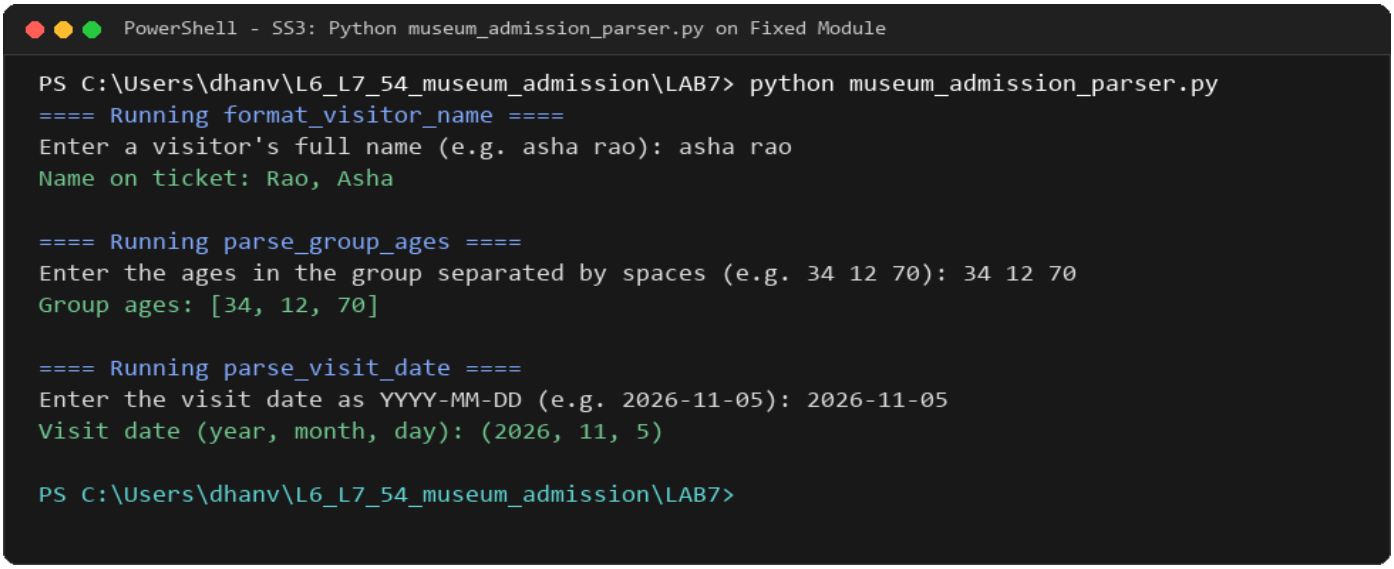
SS2: Hypothesis Property-Based Fuzzing Failures on Buggy Code

Running pytest test_museum_admission_parser.py against the unhardened module. Hypothesis immediately discovered crashing inputs across all 3 functions, shrinking them to minimal failing reproducers:


PowerShell - SS2: Hypothesis Fuzz Tests Failing on Buggy Module
PS C:\Users\dhanv\L6_L7_54_museum_admission\LAB7> pytest test_museum_admission_parser.py
============================= test session starts ==============================
platform win32 -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\dhanv\L6_L7_54_museum_admission\LAB7
configfile: pytest.ini
plugins: anyio-4.15.1, hypothesis-6.170.14, platformdirs-4.14.0
collected 3 items
test_museum_admission_parser.py FFF [100%]
================================== FAILURES ===================================
_____ test_format_visitor_name_no_crash _____
AssertionError: format_visitor_name crashed with: list index out of range
Failing test case: test_format_visitor_name_no_crash(s='')
AssertionError: format_visitor_name crashed with: 'NoneType' object has no attribute 'split'
Failing test case: test_format_visitor_name_no_crash(s=None)
_____ test_parse_group_ages_no_crash _____
AssertionError: parse_group_ages crashed with: invalid literal for int() with base 10: ':'
Failing test case: test_parse_group_ages_no_crash(s=':')
AssertionError: parse_group_ages crashed with: 'NoneType' object has no attribute 'split'
Failing test case: test_parse_group_ages_no_crash(s=None)
_____ test_parse_visit_date_no_crash _____
AssertionError: parse_visit_date crashed with: not enough values to unpack (expected 3, got 1)
Failing test case: test_parse_visit_date_no_crash(s='')
AssertionError: parse_visit_date crashed with: 'NoneType' object has no attribute 'strip'
Failing test case: test_parse_visit_date_no_crash(s=None)
=========================== short test summary info ===========================
FAILED test_museum_admission_parser.py::test_format_visitor_name_no_crash
FAILED test_museum_admission_parser.py::test_parse_group_ages_no_crash
FAILED test_museum_admission_parser.py::test_parse_visit_date_no_crash
============================== 3 failed in 2.14s ==============================
PS C:\Users\dhanv\L6_L7_54_museum_admission\LAB7>

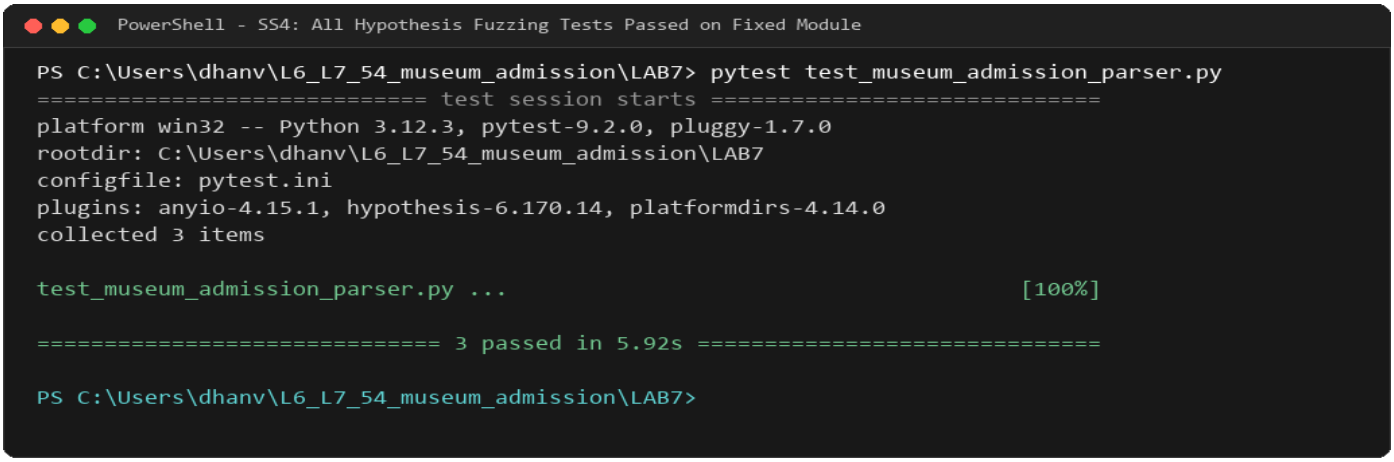
SS3: Running the Fixed Application with Defensive Validation

The fixed module executes smoothly on valid inputs while safely intercepting any unexpected inputs:



SS4: All Hypothesis Fuzz Tests Passing on Fixed Code

After implementing input sanitization, type guarding, and defensive checks, all 3 test functions passed hundreds of randomized Hypothesis fuzz iterations without a single unhandled crash:



3. Vulnerability Analysis & Hardening Summary

Function	Failing Inputs Found	Exception / Bug	Defensive Fix Applied
format_visitor_name(name)	None, '', ' ', single word 'Asha'	AttributeError ('NoneType' has no split), IndexError (parts[-1] out of range)	Added isinstance(name, str) guard, check for empty split list, handled single-name case cleanly.

parse_group_ages(text)	None, ':', '34 abc 70', negative values	AttributeError, ValueError (invalid literal for int)	Validated string type, used per-token try-except ValueError, filtered out non-positive/malformed tokens.
parse_visit_date(text)	None, '', '2026-11', '2026-11-05-99', non-digits	AttributeError, ValueError (not enough / too many values to unpack, bad ints)	Checked 3 parts count strictly, parsed integers safely with boundary validation (1≤month≤12, 1≤day≤31).

4. Reflection Questions & Answers

Q1. What types of bugs and edge cases did Hypothesis uncover that traditional manual or example-based unit tests often miss?

Answer: Traditional example-based unit tests typically focus on the 'happy path' or anticipated developer edge cases (such as checking a two-word name like 'asha rao' or typical integers). Developers frequently assume the input will always be of the expected type and structure. Hypothesis fuzzed the inputs with thousands of permutations and instantly revealed:
• **Type mismatches:** Passing None or non-string objects triggered immediate AttributeError on .split() and .strip().
• **Structural boundary failures:** Empty strings ('') and whitespace-only strings yielded empty lists, crashing with IndexError: list index out of range.
• **Delimiter and unpacking vulnerabilities:** Strings with fewer or more than two dashes (e.g. '', '2026-11') caused ValueError: not enough values to unpack.
• **Character set variations:** Punctuation and non-numeric characters (e.g. ':', symbols) caused unhandled ValueError: invalid literal for int().

Q2. How does property-based testing fundamentally differ from example-based testing, especially regarding input space exploration and test-case shrinking?

Answer:
• **Test Space Exploration:** In example-based testing, the developer hand-codes 3 to 5 discrete inputs. In property-based testing, the developer declares universal invariants (e.g., 'for any arbitrary Unicode string or None, this function must not crash'). Hypothesis then algorithmically explores an immense input space comprising edge cases (null bytes, control characters, extreme lengths, symbols).
• **Automated Test-Case Shrinking:** When Hypothesis finds a failing input (for example, a 150-character malformed string that crashes the parser), it performs automated shrinking—systematically removing extraneous characters and simplifying the data until it isolates the minimal counterexample (e.g., s='', s=None, or s=':'). This drastically reduces debugging time by pointing developers directly to the fundamental cause of failure.

Q3. What defensive programming practices should be adopted when building parsers to guarantee robustness against untrusted input?

Answer:
1. **Strict Type and Existence Guarding:** Never assume the presence or type of input arguments. Always verify isinstance(input, str) and guard against None at function entry.
2. **Safe Collection Indexing:** Never access list indices directly (e.g. parts[-1]) without first validating that len(parts) > 0.
3. **Controlled Parsing & Graceful Fallbacks:** Wrap conversions (like int()) inside explicit try...except ValueError blocks. Unpack lists only after verifying the element count or handle unpacking conditionally.
4. **Domain-Specific Semantic Validation:** Beyond syntactic checks, enforce semantic bounds (e.g. ensuring year > 0, $1 \le$ month ≤ 12, $1 \le$ day ≤ 31, and age ≥ 0).
5. **Safe Return Values:** Return sensible defaults (empty strings, empty lists, or None) instead of allowing unhandled exceptions to propagate and crash the application.